CIS 568 · DATA VISUALIZATION · FINAL PROJECT · ADDENDUM

Technology Selection Rationale for Map Rendering

Why Leaflet was chosen over deck.gl for the NTSB Aviation Accident Visualization

---

Michael J. Evan

Graduate Computer Science Department
University of Massachusetts Dartmouth

SPRING 2026

Summary

> This document presents the rationale for selecting **Leaflet** over **deck.gl** as the mapping library for an interactive visualization of approximately 170,000 NTSB aviation accident records (1962–present), deployed as a static site via GitHub Pages.

deck.gl was evaluated due to its GPU-accelerated rendering capabilities for large-scale geospatial data. However, there was no deck.gl requirement listed in the syllabus for the final project, so Leaflet was selected based on:

- equivalent basemap rendering for raster tile services
- broader client hardware accessibility
- compatibility with static hosting
- sufficient performance for the project's requirements

> **This was a deliberate architectural decision, not a capability limitation.**

Library Overview

Leaflet is a lightweight JavaScript mapping library that renders via DOM and HTML5 Canvas (CPU-bound). It requires no build pipeline, has an extensive plugin ecosystem including marker clustering, and supports canvas-accelerated rendering.

deck.gl is a WebGL-powered framework for large-scale data visualization. It renders directly on the GPU, enabling smooth interaction with million-point datasets. It typically requires a JavaScript bundler and is commonly paired with Mapbox GL or MapLibre for basemaps.

Basemap Rendering: Library-Agnostic

The visualization uses Carto's Voyager and Dark Matter basemaps, accessed as raster tile endpoints:

```
https://{s}.basemaps.cartocdn.com/rastertiles/voyager/{z}/{x}/{y}.png
https://{s}.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}.png
```

Both libraries consume these identically — Leaflet via `L.tileLayer()`, deck.gl via `TileLayer`. The tiles are pre-rendered PNGs fetched from Carto's servers.

> **Neither library provides a rendering advantage for raster basemaps.**

The same applies to FAA sectional chart overlays. These are raster products served as XYZ tiles. Both libraries simply drape these images over the map canvas; neither interprets or renders them from vector data.

A deck.gl + MapLibre implementation could render vector tiles client-side, enabling runtime style customization. This capability would have added complexity without benefit.

Client Hardware and Accessibility

deck.gl's WebGL pipeline assumes GPU availability. The following configurations may experience degraded performance or failure:

- Older integrated graphics (Intel HD, AMD APUs)
- Mobile devices with limited GPU memory
- Chromebooks with restricted WebGL contexts
- Virtual machines without GPU passthrough
- Corporate environments with WebGL disabled by policy

Cross-Platform Degradation Behavior

CONDITION	LEAFLET BEHAVIOR	DECK.GL BEHAVIOR
Slow CPU	Sluggish panning, delayed rendering	Minimal impact
Weak or no GPU	No impact	Severe lag, crashes, blank canvas
WebGL disabled	No impact	Complete failure
Low memory	Gradual slowdown	WebGL context loss, crash

> **Leaflet degrades gracefully — users experience slower performance but retain full functionality.**

> **deck.gl fails catastrophically — when GPU resources are unavailable, the visualization becomes unusable or completely blank.**

The target audience includes aviation safety researchers, students, pilots, and general users who cannot be assumed to have high-performance hardware. Leaflet ensures functionality across the widest range of client configurations, consistent with accessible design principles.

Deployment Constraints

The project was deployed via GitHub Pages as a static site — HTML, CSS, and JavaScript served without server-side processing or build steps.

Leaflet operates natively in this environment via CDN script include. deck.gl typically requires npm integration and a bundler (Webpack, Vite) to produce browser-compatible output. While deck.gl can load from CDN, this sacrifices tree-shaking, inflates bundle size, and complicates dependency management.

ASPECT	LEAFLET	DECK.GL
CDN usage	Native	Suboptimal
Bundler required	No	Typically yes
Setup time	Minutes	Hours

Performance Assessment

DATASET SIZE	LEAFLET (CANVAS / CLUSTER)	DECK.GL (WEBGL)
10,000 points	Smooth	Smooth
50,000 points	Acceptable	Smooth
170,000 points	Acceptable with clustering	Smooth
1,000,000+ points	Degraded	Smooth

For 170,000 static records with marker clustering, Leaflet provided acceptable performance. The visualization does not require real-time animation, streaming data, or GPU-accelerated aggregation — scenarios where deck.gl's overhead yields significant return.

Acknowledged Tradeoffs

1. **Scalability ceiling.** Datasets exceeding 1M records or requiring real-time animation would necessitate deck.gl.
2. **Visual density.** deck.gl produces smoother rendering at extreme point densities (heatmaps, hexbins).
3. **Future extensibility.** Temporal animations or 3D terrain would warrant migration to deck.gl.

These tradeoffs were acceptable given project scope and audience.

Conclusion

Leaflet was selected over deck.gl based on:

1. **Equivalent basemap rendering** — raster tiles are library-agnostic.
2. **Client accessibility** — CPU-based rendering with graceful degradation versus GPU-dependent catastrophic failure.
3. **Deployment compatibility** — zero-build-step architecture for static hosting.
4. **Sufficient performance** — clustering handled 170k records adequately.
5. **Scope alignment** — focus on visualization design, not rendering infrastructure.

deck.gl is appropriate for million-point datasets, real-time streaming, or GPU-accelerated aggregation. For this project's requirements, Leaflet provided equivalent functionality with broader accessibility and lower implementation overhead.

References

- Leaflet: https://leafletjs.com/
- deck.gl: https://deck.gl/
- Carto Basemaps: https://carto.com/basemaps/
- Munzner, T. (2014). *Visualization Analysis and Design.* CRC Press.